

条款 33：避免遮掩继承而来的名称

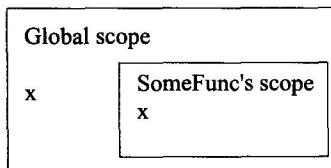
Avoid hiding inherited names.

关于“名称”，莎士比亚说过这样一句话：“名称是什么呢？”他问，“一朵玫瑰叫任何名字还是一样芬芳。”吟游诗人也写过这样的话：“偷了我的好名字的人呀……害我变得好可怜。”完全正确。这把我们引到了 C++ “继承而来的名称”。

这个题材和继承其实无关，而是和作用域（scopes）有关。我们都知道在诸如这般的代码中：

```
int x;                      //global 变量
void someFunc()
{
    double x;               //local 变量
    std::cin >> x;          //读一个新值赋予 local 变量 x
}
```

这个读取数据的语句指涉的是 local 变量 x，而不是 global 变量 x，因为内层作用域的名称会遮掩（遮蔽）外围作用域的名称。我们可以这样看本例的作用域形势：



当编译器处于 `someFunc` 的作用域内并遭遇名称 `x` 时，它在 local 作用域内查找是否有东西带着这个名称。如果找到就不再找其他作用域。本例的 `someFunc` 的 `x` 是 `double` 类型而 `global x` 是 `int` 类型，但那不要紧。C++ 的名称遮掩规则（name-hiding rules）所做的唯一事情就是：遮掩名称。至于名称是否应和相同或不同的类型，并不重要。本例中一个名为 `x` 的 `double` 遮掩了一个名为 `x` 的 `int`。

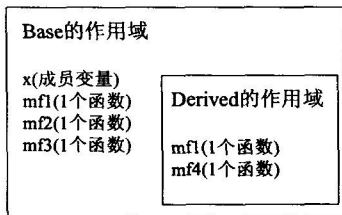
现在导入继承。我们知道，当位于一个 `derived class` 成员函数内指涉（refer to）`base class` 内的某物（也许是个成员函数、`typedef`、或成员变量）时，编译器可以找出我们所指涉的东西，因为 `derived classes` 继承了声明于 `base classes` 内的所有东西。实际运作方式是，`derived class` 作用域被嵌套在 `base class` 作用域内，像这样：

```

class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf2();
    void mf3();
    ...
};

class Derived: public Base {
public:
    virtual void mf1();
    void mf4();
    ...
};

```



此例内含一组混合了 `public` 和 `private` 名称, 以及一组成员变量和成员函数名称。这些成员函数包括 `pure virtual`, `impure virtual` 和 `non-virtual` 三种, 这是为了强调我们谈的是名称, 和其他无关。这个例子也可以加入各种名称类型, 例如 `enums`, `nested classes` 和 `typedefs`。整个讨论中唯一重要的是这些东西的名称, 至于这些东西是什么并不重要。本例使用单一继承, 然而一旦了解单一继承下发生的事, 很容易就可以推想 C++ 在多重继承下的行为。

假设 `derived class` 内的 `mf4` 的实现码部分像这样:

```

void Derived::mf4( )
{
    ...
    mf2();
    ...
}

```

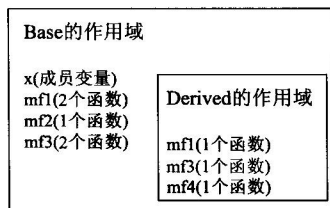
当编译器看到这里使用名称 `mf2`, 必须估算它指涉 (*refer to*) 什么东西。编译器的做法是查找各作用域, 看看有没有某个名为 `mf2` 的声明式。首先查找 `local` 作用域 (也就是 `mf4` 覆盖的作用域), 在那儿没找到任何东西名为 `mf2`。于是查找其外围作用域, 也就是 `class Derived` 覆盖的作用域。还是没找到任何东西名为 `mf2`, 于是再往外围移动, 本例为 `base class`。在那儿编译器找到一个名为 `mf2` 的东西了, 于是停止查找。如果 `Base` 内还是没有 `mf2`, 查找动作便继续下去, 首先找内含 `Base` 的那个 `namespace(s)` 的作用域 (如果有的话), 最后往 `global` 作用域找去。

刚才我描述的程序虽然精确，但范围不够广。我们的目标并不是为了知道撰写编译器必须实践的名称查找规则，而是希望知道足够的信息，用以避免发生让人不快的惊讶。对于后者，现在我们有了丰富的信息。

再次考虑前一个例子，这次让我们重载 `mf1` 和 `mf3`，并且添加一个新版 `mf3` 到 `Derived` 去。如条款 36 所说，这里发生的事情是：`Derived` 重载了 `mf3`，那是一个继承而来的 `non-virtual` 函数。这会使整个设计立刻显得疑云重重，但为了充分认识继承体系内的“名称可视性”，我们暂时安之若素。

```
class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
    virtual void mf2();
    void mf3();
    void mf3(double);
    ...
};

class Derived: public Base {
public:
    virtual void mf1();
    void mf3();
    void mf4();
    ...
};
```



这段代码带来的行为会让每一位第一次面对它的 C++ 程序员大吃一惊。以作用域为基础的“名称遮掩规则”并没有改变，因此 `base class` 内所有名为 `mf1` 和 `mf3` 的函数都被 `derived class` 内的 `mf1` 和 `mf3` 函数遮掩掉了。从名称查找观点来看，`Base::mf1` 和 `Base::mf3` 不再被 `Derived` 继承！

```
Derived d;
int x;
...
d.mf1();           //没问题，调用 Derived::mf1
d.mf1(x);          //错误！因为 Derived::mf1 遮掩了 Base::mf1
d.mf2();           //没问题，调用 Base::mf2
d.mf3();           //没问题，调用 Derived::mf3
d.mf3(x);          //错误！因为 Derived::mf3 遮掩了 Base::mf3
```

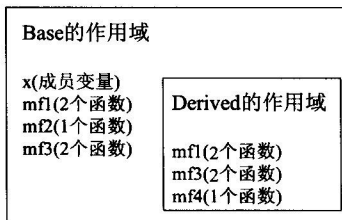
如你所见, 上述规则都适用, 即使 `base classes` 和 `derived classes` 内的函数有不同的参数类型也适用, 而且不论函数是 `virtual` 或 `non-virtual` 一体适用。这和本条款一开始展示的道理相同, 当时函数 `someFunc` 内的 `double x` 遮掩了 `global` 作用域内的 `int x`, 如今 `Derived` 内的函数 `mf3` 遮掩了一个名为 `mf3` 但类型不同的 `Base` 函数。

这些行为背后的基本理由是为了防止你在程序库或应用框架 (`application framework`) 内建立新的 `derived class` 时附带地从疏远的 `base classes` 继承重载函数。不幸的是你通常会想继承重载函数。实际上如果你正在使用 `public` 继承而又不继承那些重载函数, 就是违反 `base` 和 `derived classes` 之间的 **is-a** 关系, 而条款 32 说过 **is-a** 是 `public` 继承的基石。因此你几乎总会想要推翻 (`override`) C++ 对“继承而来的名称”的缺省遮掩行为。

你可以使用 `using` 声明式达成目标:

```
class Base {
private:
    int x;
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
    virtual void mf2();
    void mf3();
    void mf3(double);
    ...
};

class Derived: public Base {
public:
    using Base::mf1;    //让 Base class 内名为 mf1 和 mf3 的所有东西
    using Base::mf3;    //在 Derived 作用域内都可见 (并且 public)
    virtual void mf1();
    void mf3();
    void mf4();
    ...
};
```



现在, 继承机制将一如往昔地运作:

```

Derived d;
int x;
...
d.mf1();      //仍然没问题, 仍然调用 Derived::mf1
d.mf1(x);     //现在没问题了, 调用 Base::mf1
d.mf2();      //仍然没问题, 仍然调用 Base::mf2
d.mf3();      //没问题, 调用 Derived::mf3
d.mf3(x);     //现在没问题了, 调用 Base::mf3

```

这意味如果你继承 **base class** 并加上重载函数, 而你又希望重新定义或覆写 (推翻) 其中一部分, 那么你必须为那些原本会被遮掩的每个名称引入一个 **using** 声明式, 否则某些你希望继承的名称会被遮掩。

有时候你并不想继承 **base classes** 的所有函数, 这是可以理解的。在 **public** 继承下, 这绝对不可能发生, 因为它违反了 **public** 继承所暗示的“**base** 和 **derived classes** 之间的 **is-a** 关系”。(这也就是为什么上述 **using** 声明式被放在 **derived class** 的 **public** 区域的原因: **base class** 内的 **public** 名称在 **publicly derived class** 内也应该是 **public**。)然而在 **private** 继承之下 (见条款 39) 它却可能是有意义的。例如假设 **Derived** 以 **private** 形式继承 **Base**, 而 **Derived** 唯一想继承的 **mf1** 是那个无参数版本。**using** 声明式在这里派不上用场, 因为 **using** 声明式会令继承而来的某给定名称之所有同名函数在 **derived class** 中都可见。不, 我们需要不同的技术, 即一个简单的转交函数 (**forwarding function**):

```

class Base {
public:
    virtual void mf1() = 0;
    virtual void mf1(int);
    ...                //与前同
};

class Derived: private Base {
public:
    virtual void mf1() //转交函数 (forwarding function);
    { Base::mf1( ); } //暗自成为 inline (见条款 30)
    ...
};

...
Derived d;
int x;
d.mf1();                //很好, 调用的是 Derived::mf1
d.mf1(x);               //错误! Base::mf1() 被遮掩了

```

`inline` 转交函数 (forwarding function) 的另一个用途是为那些不支持 `using` 声明式 (注: 这并非正确行为) 的老旧编译器另辟一条新路, 将继承而得的名称汇入 `derived class` 作用域内。

这就是继承和名称遮掩的完整故事。但是当继承结合 `templates`, 我们又将面对“继承名称被遮掩”的一个全然不同的形式。关于“以角括号定界”的所有东西, 详见条款 43。

请记住

- `derived classes` 内的名称会遮掩 `base classes` 内的名称。在 `public` 继承下从来没有人希望如此。
- 为了让被遮掩的名称再见天日, 可使用 `using` 声明式或转交函数 (forwarding functions)。

条款 34: 区分接口继承和实现继承

Differentiate between inheritance of interface and inheritance of implementation.

表面上直截了当的 `public` 继承概念, 经过更严密的检查之后, 发现它由两部分组成: 函数接口 (function interfaces) 继承和函数实现 (function implementations) 继承。这两种继承的差异, 很像本书导读所讨论的函数声明与函数定义之间的差异。

身为 `class` 设计者, 有时候你会希望 `derived classes` 只继承成员函数的接口 (也就是声明); 有时候你又会希望 `derived classes` 同时继承函数的接口和实现, 但又希望能够覆写 (override) 它们所继承的实现; 又有时候你希望 `derived classes` 同时继承函数的接口和实现, 并且不允许覆写任何东西。

为了更好地感觉上述选择之间的差异, 让我们考虑一个展现绘图程序中各种几何形状的 `class` 继承体系:

```
class Shape {
public:
    virtual void draw( ) const = 0;
    virtual void error(const std::string& msg);
    int objectID( ) const;
    ...
};

class Rectangle: public Shape { ... };
class Ellipse: public Shape { ... };
```

Shape 是个抽象 class；它的 pure virtual 函数 draw 使它成为一个抽象 class。所以客户不能够创建 Shape class 的实体，只能创建其 derived classes 的实体。尽管如此，Shape 还是强烈影响了所有以 public 形式继承它的 derived classes，因为：

- 成员函数的接口总是会被继承。一如条款 32 所说，public 继承意味 **is-a**（是一种），所以对 base class 为真的任何事情一定也对其 derived classes 为真。因此如果某个函数可施行于某 class 身上，一定也可施行于其 derived classes 身上。

Shape class 声明了三个函数。第一个是 draw，于某个隐喻的视屏中画出当前对象。第二个是 error，准备让那些“需要报导某个错误”的成员函数调用。第三个是 objectID，返回当前对象的一个独一无二的整数识别码。每个函数的声明方式都不相同：draw 是个 pure virtual 函数；error 是个简朴的（非纯）impure virtual 函数；objectID 是个 non-virtual 函数。这些不同的声明带来什么样的暗示呢？

首先考虑 pure virtual 函数 draw：

```
class Shape {  
public:  
    virtual void draw( ) const = 0;  
    ...  
};
```

pure virtual 函数有两个最突出的特性：它们必须被任何“继承了它们”的具象 class 重新声明，而且它们在抽象 class 中通常没有定义。把这两个性质摆在一起，你就会明白：

- 声明一个 pure virtual 函数的目的是为了 let derived classes 只继承函数接口。

这对 Shape::draw 函数是再合理不过的事了，因为所有 Shape 对象都应该是可绘出的，这是合理的要求。但 Shape class 无法为此函数提供合理的缺省实现，毕竟椭圆形绘法迥异于矩形绘法。Shape::draw 的声明式乃是对具象 derived classes 设计者说，“你必须提供一个 draw 函数，但我不干涉你怎么实现它。”

令人意外的是，我们竟然可以为 pure virtual 函数提供定义。也就是说你可以为 Shape::draw 供应一份实现代码，C++ 并不会发出怨言，但调用它的唯一途径是“调用时明确指出其 class 名称”：

```
Shape* ps = new Shape;           //错误! Shape 是抽象的
Shape* ps1 = new Rectangle;      //没问题
ps1->draw();                      //调用 Rectangle::draw
Shape* ps2 = new Ellipse;        //没问题
ps2->draw();                     //调用 Ellipse::draw
ps1->Shape::draw();              //调用 Shape::draw
ps2->Shape::draw();              //调用 Shape::draw
```

除了能够帮助你在鸡尾酒派对上留给大师级程序员一个深刻的印象,一般而言这项性质用途有限。但是一如稍后你将看到,它可以实现一种机制,为简朴的(非纯) `impure virtual` 函数提供更平常更安全的缺省实现。

简朴的 `impure virtual` 函数背后的故事和 `pure virtual` 函数有点不同。一如往常, `derived classes` 继承其函数接口,但 `impure virtual` 函数会提供一份实现代码, `derived classes` 可能覆写 (*override*) 它。稍加思索,你就会明白:

- 声明简朴的(非纯) `impure virtual` 函数的目的,是让 `derived classes` 继承该函数的接口和缺省实现。

考虑 `Shape::error` 这个例子:

```
class Shape {
public:
    virtual void error(const std::string& msg);
    ...
};
```

其接口表示,每个 `class` 都必须支持一个“当遇上错误时可调用”的函数,但每个 `class` 可自由处理错误。如果某个 `class` 不想针对错误做出任何特殊行为,它可以退回到 `Shape` `class` 提供的缺省错误处理行为。也就是说 `Shape::error` 的声明式告诉 `derived classes` 的设计者,“你必须支持一个 `error` 函数,但如果你不想自己写一个,可以使用 `Shape` `class` 提供的缺省版本”。

但是,允许 `impure virtual` 函数同时指定函数声明和函数缺省行为,却有可能造成危险。欲探讨原因,让我们考虑 XYZ 航空公司设计的飞机继承体系。该公司只有 A 型和 B 型两种飞机,两者都以相同方式飞行。因此 XYZ 设计出这样的继承体系: